

Avaliação Empírica da Qualidade e dos Defeitos em Software Gerado por Agentes Gratuitos de IA para Código: Um Estudo Exploratório sob Requisitos Controlados

Versão: pré-print v0.1 (2026-09-06)

Autor: Bruno Bertin Marquez · ORCID: [0009-0005-3546-8302](https://orcid.org/0009-0005-3546-8302) (<https://orcid.org/0009-0005-3546-8302>) · Pesquisador independente — Pós-graduado em Gestão da Qualidade de Software; Pós-graduando em Engenharia de Requisitos.

Resumo

Software de código-fonte gerado por modelos de linguagem de grande escala (LLMs) tornou-se rotina no ciclo de Engenharia de *Software* (ES). A literatura demonstrou, em fragmentos isolados, que esse código apresenta defeitos com padrões característicos, vulnerabilidades de segurança frequentes, acúmulo de dívida técnica, testes com efetividade limitada e reprodutibilidade imperfeita. Contudo, nenhum estudo revisado combina, na mesma cadeia experimental controlada, a geração de **sistemas completos** por diferentes **agentes de IA** a partir do **mesmo requisito**, sob **múltiplas execuções**, avaliados por um **oráculo independente** com cobertura simultânea de funcionalidade, defeitos, segurança, manutenibilidade e reprodutibilidade. Este artigo apresenta um experimento controlado, prospectivo e reproduzível que preenche essa lacuna: quatro agentes (modelos distintos executados no mesmo *framework* de agente `opencode`, 2× Nemotron, Ling e Mimo) produziram, cada um, três execuções independentes de um sistema completo (API REST de gerenciamento de tarefas em Python/FastAPI/PostgreSQL), totalizando 12 entregas avaliadas por uma suíte oráculo independente de 81 testes, mais verificações não funcionais (segurança, manutenibilidade, reprodutibilidade, performance) e uma matriz de defeitos classificada segundo a taxonomia de Tambon et al. com severidades e concordância inter-avaliador. **Este estudo foca exclusivamente em combinações agente/modelo gratuitamente disponíveis (free-tier); os resultados caracterizam essa camada e não devem ser generalizados para modelos premium/fronteira sem estudo adicional.** Resultados: apenas 3 das 12 entregas foram bootáveis; os 12 defeitos Tambon foram 100% Blocker ou Critical; os testes gerados pelos próprios agentes não detectaram nenhum dos defeitos (0/12), enquanto o oráculo detectou 4/4 dos alcançáveis (RQ6/H3); padrões reincidentes (dependência ausente; incompatibilidade *passlib+bcrypt*; configuração de migração; versão "fantasma") concentram a maioria dos defeitos e se repetem entre modelos "free", sugerindo um "vale comum" de qualidade no momento da coleta. O estudo sustenta, descritivamente, que a aprovação pelos próprios testes do agente não é substituta da validação independente, e que um índice único colapsaria dimensões de qualidade (funcional ≠ global). O poder estatístico é limitado ($n=12$; células esperadas <5), de modo que H_1-H_3 são apresentadas como tendências, não conclusões; um estudo complementar de laboratório (não evidenciário) reforça a separação entre *detectar* e *testar*.

Palavras-chave: *software* gerado por IA; agentes gratuitos de código; qualidade de *software*; defeitos; oráculo independente; testes de *software*; Engenharia de *Software*; experimento controlado; estudo exploratório.

1. Introdução

Os modelos de linguagem de grande escala (LLMs) permeiam hoje praticamente todo o ciclo de Engenharia de *Software*, com concentração em geração de código, geração de testes e correção de

programas [A01]. Revisões sistemáticas consolidadas apontam limitações recorrentes — alucinações, vulnerabilidades de segurança, baixa generalização e problemas de interpretabilidade [A02]. Esse campo, contudo, apresenta um conflito de resultados que permanece em aberto: há evidência *favorável* à IA (experimento controlado do GitHub com Copilot [D03]; menor esforço de correção estimado em alguns cenários [D02]) e evidência *desfavorável* (perfis de defeitos distintos em larga escala [B03]; vulnerabilidades frequentes [E01–E03]). A ausência de consenso é justamente o que justifica uma investigação empírica controlada.

No plano dos **defeitos**, estudos empíricos mostraram que o código gerado por LLMs possui padrões característicos e recorrentes. Tambon et al. [B01] analisaram 333 bugs de CodeGen, PanGu-Coder e Codex e propuseram uma taxonomia de **dez padrões** de defeito (*misinterpretation*, *syntax error*, *silly mistake*, *prompt-biased code*, *missing corner case*, *wrong input type*, *hallucinated object*, *wrong attribute*, *incomplete generation*, *non-prompted consideration*), validada por 34 pesquisadores e profissionais. Em escala muito maior, Cotroneo, Improta e Liguori [B03] compararam mais de 500 mil amostras de código humano e de IA e concluíram que **IA e humanos têm perfis de defeitos diferentes** — a IA tende a mais *constructs* não utilizados, *debugging hardcoded* e vulnerabilidades de segurança de alto risco. Além disso, resultados de LLMs são **não-deterministas**: a mesma entrada pode produzir saídas diferentes, afetando a correção e a reprodutibilidade científica [B04].

No plano dos **testes**, a literatura é igualmente cautelosa. Testes gerados por LLMs podem conter *test smells* mesmo quando válidos [C03]; sua capacidade de detecção de defeitos é limitada (29–60% dos defeitos detectáveis em projetos Defects4J [C04]); e métricas tradicionais de cobertura e *mutation score* perdem confiabilidade quando o código sob teste contém um bug [C05]. Modelos distintos tendem a omitir testes de valores especiais (*None*, *inf*, *NaN*) [C06] — um eco direto da categoria *missing corner case* da taxonomia de Tambon [B01].

No plano da **qualidade não funcional**, a literatura privilegia a correção funcional e subavalia segurança, performance e, sobretudo, manutenibilidade e dívida técnica [D04]; código de IA acumula *code smells* que persistem no tempo [D05]. No plano da **segurança**, rupturas são frequentes: de 29,5% (Python) a 24,2% (JavaScript) de snippets vulneráveis [E01]; todos os LLMs analisados gerando vulnerabilidades, muitas de severidade alta ou crítica [E02]; e estratégias de *prompting* de segurança que mudam a distribuição, mas não reduzem de forma significativa a densidade de vulnerabilidades [E03].

No plano dos **agentes de programação**, o campo migrou de "LLM responde a pergunta" para "agente executa tarefas reais de desenvolvimento". Esses agentes executam refatorações [F01], produzem projetos com reprodutibilidade limitada — apenas 68,3% executam imediatamente em ambiente limpo, com expansão média de 13,5× entre dependências declaradas e necessárias [F02] — e falham de forma desigual entre si: a taxa de reversão de *commits* varia de 0,7% (Codex) a 7,6% (Copilot) [F05].

A lacuna. Os fragmentos acima foram demonstrados de forma isolada. Porém, nenhum estudo revisado (0/25 na nossa revisão — ver §3) une essas dimensões em um **único experimento controlado e reproduzível** com a mesma especificação → diferentes agentes → sistemas completos → ≥ 3 execuções → oráculo independente → defeitos + segurança + manutenibilidade + reprodutibilidade → análise estatística (lacunas G1–G5). Estudos retrospectivos observam dados reais (e.g., [D05], [F04], [F05]), mas não executam um experimento com requisito e oportunidades idênticos entre os grupos; a maioria dos estudos trabalha com *snippets*, funções ou *patches*, não com **sistemas completos**; e raros separam quem **gera** de quem **avalia** com um oráculo independente da IA geradora.

Problema de pesquisa. Redigido com a devida neutralidade:

Embora a literatura demonstre que código gerado por IA apresenta defeitos característicos, vulnerabilidades frequentes e testes com baixa capacidade de detecção, **não há evidência empírica controlada sobre como diferentes agentes gratuitos de IA para código se comparam quando produzem sistemas completos a partir dos mesmos requisitos**, avaliados por um **oráculo independente**, sob **múltiplas execuções**, cobrindo simultaneamente **qualidade funcional, defeitos, segurança, manutenibilidade e reprodutibilidade**.

Objetivo geral. Avaliar empiricamente a qualidade e os defeitos presentes em sistemas de *software* produzidos por diferentes agentes/modelos de IA sob condições experimentais controladas (mesmo requisito, mesmo prompt, múltiplas execuções, oráculo independente). Como objetivos específicos: comparar agentes na geração a partir do mesmo requisito; identificar e classificar defeitos (taxonomia Tambon + severidade); avaliar efetividade de testes sem confiar isoladamente em cobertura/mutação; avaliar qualidade estrutural (ISO/IEC 25010), segurança (CWE/OWASP) e reprodutibilidade; investigar a capacidade da própria IA de detectar seus próprios defeitos; e comparar os resultados estatisticamente entre agentes.

Contribuições. (i) Primeiro experimento controlado, prospectivo e reproduzível — até onde a revisão alcança — que combina, na mesma cadeia, sistemas completos + defeitos + segurança + manutenibilidade + reprodutibilidade + múltiplas execuções + oráculo independente; (ii) evidência empírica de padrões de defeito **reincidentes e transferíveis entre modelos "free"** de um mesmo pipeline de agente, incluindo um "vale comum" de qualidade; (iii) evidência sobre a **testabilidade** dos sistemas gerados (RQ6/H3): os testes do próprio agente não detectaram nenhum defeito, enquanto o oráculo independente detectou todos os alcançáveis. **Escopo:** este estudo caracteriza agentes de código **free-tier**; não deve ser generalizado para modelos premium/fronteira sem estudo adicional.

2. Trabalhos Relacionados

Condensamos a revisão em grupos temáticos; fichas completas com DOI encontram-se em §8. Detalhes do método de revisão e das 25 fontes estão documentados nos artefatos da pesquisa (matriz e revisão bibliográfica).

A — LLMs em Engenharia de Software. Hou et al. [A01] conduziram a maior SLR do campo (395 artigos, 85 tarefas de SE em seis atividades), mapeando modelos, *datasets*, estratégias de *prompting* e desafios de avaliação. Umama et al. [A02] revisaram 58 estudos (PRISMA) e elencaram limitações recorrentes (alucinação, segurança, generalização, interpretabilidade). Essas SLRs fornecem nosso vocabulário de RQs, mas não conduzem experimento controlado comparando agentes na geração de sistemas completos.

B — Defeitos em código gerado por LLMs. Tambon et al. [B01] consolidaram a taxonomia de dez padrões que adotamos para classificar defeitos. Dou et al. [B02] mostraram que bugs variam entre *benchmarks* e situações reais, e que a autocorreção melhora a aprovação (+29,2%), sinalizando uma variável (revisão pela própria IA) que deixamos **fora** da primeira versão do nosso desenho para manter o controle. Cotroneo et al. [B03] estabeleceram que IA e humanos têm **perfis de defeitos diferentes**, motivando-nos a comparar distribuições (perfil), não apenas totais. Ouyang et al. [B04] demonstraram o **não-determinismo**, justificando nossas ≥ 3 execuções por modelo.

C — Testes com LLMs. Wang et al. [C01] mapearam as aplicações de LLMs em testes. Beer et al. [C02] avaliaram código e testes gerados por ChatGPT/Copilot (algoritmos pequenos), achando

diferenças por modelo/linguagem — avançamos para **sistema completo** com oráculo independente. Alves et al. [CO3] mostraram *test smells* mesmo em testes válidos; Yang et al. [CO4] quantificaram a detecção limitada (29–60%); Zhao et al. [CO5] contestaram a confiabilidade de cobertura/mutação quando há bug; Walczak et al. [CO6] mostraram a omissão de valores especiais. Em conjunto, sustentam nossa dimensão C (testabilidade) e a decisão de **não** confiar em coverage isolada nem nos testes da própria IA.

D — Qualidade de software. Tosi [DO1] avaliou engines de IA e concluiu pela necessidade de supervisão especializada (base metodológica). Molison et al. [DO2] evidenciam que a IA pode ser melhor/igual/pior conforme o contexto (fomenta nossa neutralidade). O experimento controlado do GitHub [DO3] favoreceu a IA em endpoints web, impedindo conclusão precipitada contra IA. Sun et al. [DO4] defenderam a adoção de ISO/IEC 25010 e mostraram a subavaliação dos aspectos não funcionais — adotamos ISO/IEC 25010 como referência. Liu et al. [DO5] quantificaram a dívida técnica (89,1% de *code smells*) em commits de IA.

E — Segurança. Fu et al. [EO1] e Morkonda et al. [EO2] quantificaram vulnerabilidades; Kharm et al. [EO3] mostraram que *prompting* de segurança não reduz significativamente a densidade. Embora nosso oráculo inclua camada de segurança, o *prompt* entregue aos agentes **não** instrui sobre segurança (para não induzir os grupos de forma desigual).

F — Agentes de programação. Horikawa et al. [FO1] distinguiram modelo de agente (relevante para nosso recorte: comparamos modelos no mesmo *framework* de agente). Vangala et al. [FO2] mostraram reprodutibilidade limitada (68,3% executam) e expansão de dependências de 13,5× — nosso estudo mede a reprodutibilidade como dimensão (RQ7). Belozarov et al. [FO3] separaram erro do requisito de erro introduzido pela IA. Ehsani et al. [FO4] e Oukhay et al. [FO5] mostraram que agentes reais se comportam de modo desigual (reversões 0,7%–7,6%), reforçando a pertinência de comparar tratamentos distintos.

Lacunas (G1–G5) e posição do nosso estudo. A revisão completa que apoia este trabalho (25 artigos: 16 com DOI de periódico/conferência, 8 *preprints*, 1 relatório institucional) revela que 8% dos estudos geram **sistema completo**, 12% usam **oráculo independente**, e 0% combinam sistema completo + defeitos + segurança, tampouco executam o desenho integral (espec→agentes→≥3 execuções→oráculo→defeitos+NFR→estatística). Posicionamos este experimento exatamente nessa lacuna (G1–G5; ver artefatos de pesquisa para a análise quantitativa da matriz).

3. Metodologia

3.1 Posição metodológica, neutralidade e pré-registro

Experimento **controlado, prospectivo e reproduzível** (diferentemente de estudos retrospectivos como [DO5, FO4, FO5]). As hipóteses, RQs, plano de análise, critérios de exclusão e o princípio de neutralidade foram fixados **antes** da coleta (documentos de problema-e-hipóteses e metodologia commitados previamente no repositório de pesquisa), evidenciado por commits com timestamp no repositório (isso **não constitui pré-registro formal em plataforma pública** como OSF ou AsPredicted; pré-registro formal está planejado para o desenho ampliado, ver Trabalhos Futuros). **Neutralidade (C1):** o experimento não parte de "IA é ruim"; se os dados mostrarem IA igual ou melhor em alguma dimensão, isso é resultado, não limitação.

3.2 Desenho experimental

- **Tipo:** comparação entre grupos (um grupo por agente), com repetições dentro do grupo.

- **Unidade experimental:** um **sistema completo** (API REST) gerado por um agente a partir da especificação.
- **Tratamento (variável independente):** identidade do modelo dentro do agente.
- **Repetições: 3 execuções independentes por agente** (trata não-determinismo [Bo4] e a RQ8).
- **Blindagem:** oráculo escrito por QA independente e mantido em repositório privado, fora do alcance dos agentes (evita *data leakage*; cf. [Co6]).

Agentes (tratamentos). Para reduzir o confundimento de ferramenta, todos os tratamentos usam o **mesmo *framework* de agente** (`opencode` — mesmo *loop* de leitura/edição/teste), variando apenas o **modelo**. Quatro modelos "free", de famílias distintas (2× Nemotron, Ling, Mimo):

Modelo	ID no <code>opencode</code>
Nemotron 3 Ultra (NVIDIA)	<code>opencode/nemotron-3-ultra-free</code>
Nemotron 3.5 Lightning (NVIDIA)	<code>opencode/nemotron-3.5-lightning-free</code>
Ling 3.0 Flash	<code>opencode/ling-3.0-flash-fin-free</code>
Mimo v2.5	<code>opencode/mimo-v2.5-free</code>

A versão exata de cada modelo é registrada por execução no manifest (agentes têm versões; [Fo5] mostrou diferenças). Limitação declarada: perde-se a comparação entre ferramentas de agente (mitigada pelo *loop* idêntico do `opencode`).

3.3 Especificação controlada (o "mesmo requisito")

Sistema-alvo: **API REST de Gerenciamento de Tarefas** (escopo moderado e realista), em Python + FastAPI + PostgreSQL. Requisitos funcionais (FR1–FR8): cadastro/login com hash e JWT; CRUD de tarefas; regras de permissão (dono/admin); filtros e paginação; validação de entrada; tratamento de erros padronizado; persistência em PostgreSQL com migrações; testes automatizados na entrega. Requisitos não funcionais (NFR) ancorados em ISO/IEC 25010 [Do4] e CWE/OWASP [Eo1–Eo3]: segurança, manutenibilidade, reprodutibilidade, performance básica. A especificação é o **único** documento entregue aos agentes, em formato idêntico. Os NFR são informados só com critério de aceite comportamental, **sem** "solução", para não induzir os grupos de forma desigual.

3.4 Oráculo independente

Suíte **independente da IA geradora**, privada e imutável durante a coleta, validada contra um sistema *golden* de referência (não-IA). Camadas:

Camada	Conteúdo	Ferramenta
Funcional	Unit, integração, API cobrindo FR1–FR8 e casos de aceite	pytest, pytest-cov, httpx
Negativo	<i>Invalid input</i> , extremos (<i>None/inf/NaN</i>), permissões, autenticação	pytest parametrizado
Segurança	Autenticação/autorização, injeção, dependências	bandit, pip-audit
Estrutura	Complexidade, duplicação, <i>code smells</i> , manutenibilidade	Ruff + radon
Testabilidade	Testes do próprio agente executados contra o oráculo; taxa de detecção real	pytest

Regras: não revisado pelo agente; não exposto em prompt; cobre todos os FR/NFR; **coverage é reportado mas não tratado como evidência de qualidade** ([Co5]); vale a detecção real de

defeitos. A suíte tem **81 testes** e é exclusivamente *black-box* em relação ao backend.

Ambiente de validação. A máquina de execução é uma VM **sem virtualização aninhada**; por isso a validação foi **nativa** (PostgreSQL nativo + venv + `alembic upgrade head` + uvicorn + gating por `/health`). Artefatos Docker exigidos na entrega continuam obrigatórios e são validados estruturalmente; serão executados em contêineres na máquina de produção. Essa condicionante é reportada como limitação (§5.3).

3.5 Matriz de defeitos e classificação

Cada defeito é registrado com: ID; agente + execução; local; **categoria** (taxonomia Tambon, 10 padrões [Bo1]); **severidade** (Blocker/Critical/Major/Minor, harmonizada com CWE para segurança); **detecção** (pelo oráculo? pelos testes do agente?); evidência (teste que reproduz). **Dupla classificação independente** por dois avaliadores, com κ de Cohen e resolução de divergências em reunião. Densidade = defeitos por KLOC do projeto entregue (LOC snapshotada, sem venv/cache).

3.6 Métricas e análise estatística

Dado $n=12$, os testes estatísticos abaixo são reportados como **sinais descritivos/informativos, não testes confirmatórios de hipótese**. Mapeamento RQ→métrica→instrumento e testes pré-definidos:

RQ/H	Métrica	Teste/Instrumento
RQ1 funcional	Taxa de passagem do oráculo por execução	pytest
RQ2/H1 densidade	Defeitos/KLOC por agente	descritivo + IC de Poisson (n pequeno)
RQ3/H2 categorias	Distribuição de categorias por agente	χ^2 + V de Cramér; Fisher exato pontual
RQ4 severidade	Distribuição por severidade	descritivo; % Blocker/Critical
RQ5/H4 global	Funcional vs NFR (dimensões separadas)	comparativo por dimensão; sem índice único
RQ6/H3 testabilidade	Defeitos detectados por testes do agente vs oráculo	matriz 2×2; Fisher exato
RQ7 reprodutibilidade	Executa em ambiente limpo; instruções; dependências	verificação de executabilidade
RQ8 variabilidade	Variância entre as ≥ 3 execuções	descritiva (CV)

Nível $\alpha=0,05$. Regra honesta: χ^2 só é conclusivo com células esperadas ≥ 5 ; com células esperadas < 5 usamos Fisher exato e **reportamos como informativo, não conclusivo**. Dado o caráter exploratório do estudo (declarado no resumo), todos os p-valores são reportados **brutos, sem ajuste para múltiplas comparações**, como sinais descritivos e não como testes confirmatórios. **RQ6/H3 é o único teste reportado com intenção confirmatória** (Fisher exato em matriz 2×2, apropriado para n pequeno).

3.7 Procedimento de coleta

(1) congelar especificação v1.0 e oráculo (privados, imutáveis); (2) para cada agente, sessão nova com o mesmo prompt ("implemente a especificação; entregue sistema executável com testes"); (3) ≥ 3 sessões independentes por agente; (4) capturar versão do agente/modelo, prompt, log, repositório e testes entregues; (5) avaliar cada entrega apenas com o oráculo e a matriz (avaliadores cegos quanto ao agente quando possível); (6) registrar evidências. Critérios de exclusão: execuções **piloto**; achados de **infraestrutura** (ex.: sessão rejeitada por permissão; disco cheio da VM); achados **NFR/não-bug**

(classificados à parte). Fases: especificação → oráculo+golden → piloto (calibração) → execução completa (4×3) → classificação/análise → consolidação.

3.8 Ameaças à validade e mitigação

Data leakage do oráculo (oráculo privado/imutável); não-determinismo (≥ 3 execuções); confundimento entre agentes (mesmo requisito/prompt/ambiente); mudança de modelo durante o estudo (congelar versões); poder estatístico baixo (relatar tamanho de efeito, $n=3$ /agente); viés do avaliador (dupla classificação, κ , avaliadores cegos, pré-registro); oráculo embutir soluções (oráculo escrito da especificação e validado contra *golden*); viés de publicação (pré-registro); escopo muito fácil/difícil (especificação moderada + piloto).

4. Resultados

Executamos **12 entregas completas (4 agentes × 3 execuções)**; pilotos de calibração foram **excluídos** da análise. Tabelas e figuras acompanham este texto (figuras 1–5, geradas por script reproduzível a partir dos agregados).

4.1 Caracterização das entregas e qualidade funcional (RQ1)

Execução	Tests	Failures	Errors	Passed	Boot
lightning/e1–e2	0	0	0	0	não
lightning/e3	81	32	48	1	sim
ling/e1–e3	0	0	0	0	não
mimo/e2	81	21	48	12	sim
mimo/e3	81	21	48	12	sim
ultra/e1–e3	0	0	0	0	não

Apenas 3 das 12 entregas foram bootáveis (IC 95% Wilson: 0,07–0,49; o oráculo executou): lightning/e3, mimo/e2 e mimo/e3. Nas executáveis, a taxa de aprovação dos 81 testes foi de **1/81** (lightning) e **12/81** (mimo, duas vezes), com 32 e 21 falhas e 48 erros, respectivamente (Fig. 3). Nenhuma entrega passou 100% do oráculo.

4.2 Defeitos e densidade (RQ2/H1; RQ3/H2)

Classificamos **12 defeitos Tambon** (mais 2 achados NFR/não-bug e 2 de infraestrutura, estes excluídos). Densidade por agente:

Agente	Defeitos	KLOC	defeitos/KLOC
ultra	4	4,30	0,93
lightning	4	2,93	1,36
ling	1	1,83	0,55
mimo	3	3,27	0,92

Descritivamente, lightning apresenta a maior densidade (1,36/KLOC) e ling a menor (0,55/KLOC) (Fig. 1). Com ≤ 4 defeitos por agente, H1 é avaliada **descritivamente** (IC de Poisson largo); não há poder para afirmar significância.

Perfil por categoria (RQ3/H2):

Agente	incomplete_gen	silly_mistake	wrong_input	non_prompted	hallucinated	wrong_attr
ultra	4	0	0	0	0	0
lightning	0	1	0	1	1	1
ling	0	0	1	0	0	0
mimo	1	2	0	0	0	0

O teste χ^2 de independência agente×categoria resultou $\chi^2=25,7$, $df=15$, $p=0,041$, **V de Cramér=0,85** — associação **informativa, mas não conclusiva** (células esperadas <5; ver §3.6). O agente **ultra** concentrou 100% dos defeitos em `incomplete_generation` (Fisher exato pontual $p=0,010$ vs demais), o achado mais sustentável; ling apontou para `wrong_input_type` ($p=0,083$), mimo para `silly_mistake` ($p=0,127$), lightning sem categoria dominante. O heatmap (Fig. 2) resume a distribuição.

4.3 Severidade (RQ4)

Todos os 12 defeitos foram **Blocker (8)** ou **Critical (4)**; nenhum Major/Minor (Fig. 5). Essa concentração é esperada e deve ser lida com o **viés de detecção**: 10/12 entregas não subiram, e um deliverable não bootável tem defeito automaticamente Blocker.

4.4 Padrões recorrentes

Padrão	Ocorrências (nas 12)	Exemplos
Dependência ausente (<code>pydantic-settings / email-validator</code>)	4 (+1 no piloto)	ultra e1/e3, mimo e1
<code>passlib 1.7.4 + bcrypt ≥4.4 → 500</code> em <code>/auth/register</code>	3 (+1 no piloto)	lightning e3, mimo e2/e3
Misconfig de reprodução (alembic sem <code>script_location</code>)	1 (+1 no piloto)	ultra e2
Versão "fantasma" (<code>pydantic==2.8.4</code> inexistente)	1	lightning e1

Esses padrões são **transferíveis entre modelos "free"** do mesmo pipeline, sugerindo um "vale comum" de qualidade no momento da coleta.

4.5 Testabilidade — os testes do agente detectam os defeitos do agente? (RQ6/H3)

Matriz 2×2 (por defeito; execuções formais):

	Testes do agente: SIM	Testes do agente: NÃO	Total
Oráculo detectou: SIM	0	4	4
Oráculo detectou: NÃO (<i>não bootável</i>)	0	8	8
Total	0	12	12

Os testes gerados pelo próprio agente não detectaram nenhum dos 12 defeitos Tambon (0/12; IC 95% Wilson: 0,00–0,26; 0/18 linhas da matriz, incluindo NFR e piloto). Nas entregas em que a funcionalidade foi exercitável, o oráculo detectou **4/4** defeitos alcançáveis (lightning/e3 ×2; mimo/e2, e3), enquanto os testes do agente detectaram **0** — mesmo quando a suíte

do agente pôde rodar contra o oráculo. O padrão o-vs-4 (linha "agente detectou" toda nula) produz Fisher marginal extremo, reportado como **tendência forte, não significância**, dado o n e o viés de seleção (3/12 bootáveis). O dado transversal e robusto: **a aprovação pelos próprios testes do agente não foi, em nenhum caso, garantia de conformidade com o oráculo** — coerente com [Co3, Co4, Co5, Co6].

4.6 Concordância inter-avaliador e reprodutibilidade (RQ7)

Sobre 18 itens da matriz (dupla classificação independente e cega): **κ categoria (Tambon) = 0,54 (moderada); κ severidade = 0,91 (quase perfeita)** (Fig. 4). As 6 divergências de categoria concentram-se em defeitos de integração de dependências com múltiplas classes plausíveis (ex.: `silly_mistake` × `wrong_input_type` no conflito `passlib+bcrypt`). **Resolução de discordâncias:** as divergências foram resolvidas por discussão de consenso entre os dois avaliadores em reunião registrada; cada discordância foi documentada com a justificativa da classificação final. O κ moderado de categoria reflete ambiguidade genuína na taxonomia de Tambon quando aplicada a defeitos de integração, e não erro de classificação. Quanto à reprodutibilidade (RQ7), 3/12 entregas executaram em ambiente limpo e com instruções; as demais falharam por dependências, configuração de migração ou versão inexistente (padrões de §4.4).

4.7 Variabilidade entre execuções (RQ8)

Onde foi mensurável (mimo), o *pass rate* foi **estável** nas duas execuções (0,15). Lightning teve uma única execução executável (0,01). Ultra e ling não produziram sinal funcional; a variabilidade é qualitativa (os três defeitos do ultra são todos `incomplete_generation`).

4.8 Qualidade global — funcional ≠ não funcional (RQ5/H4)

Há divergência entre dimensões: entregas que **sobem** ainda falham em 69–80 dos 81 testes (defeitos de *auth/DB*); entregas que **não sobem** podem passar nos NFR estáticos (estrutura/reprodutibilidade estrutural). Um índice único colapsaria dimensões distintas — sustenta, descritivamente, H4.

5. Discussão

5.1 Interpretação dos achados principais

Densidade e natureza (RQ1–RQ4; H1, H2). Há diferenças descritivas de densidade (lightning 1,36; ultra 0,93; mimo 0,92; ling 0,55 defeitos/KLOC), com o achado mais sustentável no **perfil do ultra**: 100% `incomplete_generation` ($p=0,010$ Fisher). Interpretamos em termos de **perfil**, não apenas quantidade, coerente com a evidência de que IA e humanos — e, aqui, diferentes modelos — têm perfis de defeitos distintos [Bo3]. O χ^2 aponta associação ($p=0,041$, $V=0,85$), mas **informativa, não conclusiva** (n reduzido, células <5). Com $n=12$, o desenho não tem poder estatístico para testar H1 formalmente; os dados de densidade são reportados como **tendência descritiva** (lightning > ultra $\approx$ mimo > ling), sem inferência confirmatória. **H2** encontra apoio pontual (perfil ultra), sem generalização.

Severidade (RQ4). 100% Blocker/Critical é esperado e inflacionado pelo viés de detecção (10/12 não bootáveis), impedindo medir defeitos funcionais que só apareceriam em execução. Blocker: 8/12 (IC 95% Wilson: 0,39–0,84); Critical: 4/12 (IC 95% Wilson: 0,16–0,61).

Testabilidade (RQ6/H3). O resultado mais importante da pesquisa: **0/12 defeitos detectados pelos testes do próprio agente; 4/4 pelo oráculo** nos alcançáveis. Aprovação própria ≠ validação

independente — em linha com a literatura [Co3–Co6]. A assimetria metodológica (8/12 sem execução funcional do oráculo) limita a comparação e deve ser reportada.

Funcional $\neq$ global (RQ5/H4). Dimensões funcional, estrutural e de segurança divergem; índice único as colapsaria [Do4].

Padrões reincidentes. Dependências ausentes, conflito *passlib+bcrypt*, misconfig de migração e versão fantasma concentram os defeitos e se repetem entre modelos distintos — um "vale comum" de qualidade para modelos free do pipeline no momento da coleta, independentemente da família. Isso estende, para o cenário controlado e com modelos gratuitos, os achados de reprodutibilidade/executabilidade limitada [Fo2] e de confiabilidade desigual entre agentes [Fo5].

Variabilidade (RQ8). Limitada onde mensurável; qualitativa em geral.

5.2 Estudo complementar de laboratório (não evidenciário)

Ressalva metodológica: esta subseção compara modelos e tarefas **distintos** do experimento formal (chat-LLMs, função isolada; não sistemas completos). Qualquer convergência é apenas **indício**, nunca evidência cruzada. Não constitui evidência do experimento controlado.

Em paralelo, realizamos explorações com **chat-LLMs** (Oreate/Claude/Gemini — modelos diferentes dos 4 tratamentos, acesso via interface web gratuita, sem uso de API paga) em tarefas isoladas de QA sobre uma função `create_order` com gabarito de 6 defeitos (EXP-001: detectar; EXP-002: projetar testes). Achados: na **detecção** (EXP-001), dois modelos identificaram 6/6 e um 5/6 (perdendo o defeito de tipo/domínio D1), ecoando a categoria `wrong input type` [Bo1] e a omissão de casos especiais [Co6]; fronteiras explícitas (`>` vs `>=`) foram detectadas por todos. Na **geração de testes** (EXP-002), a cobertura caiu para 5/6, 4/6 e 4/6; o caso mais informativo foi D1: um modelo que **não criou teste** para a quantidade fracionária, embora tivesse **identificado** o defeito no EXP-001. Isso ilustra, em escala isolada, a mesma separação subjacente à RQ6/H3: **reconhecer uma condição defeituosa e traduzi-la em um teste capaz de revelá-la são capacidades distintas** — coerente com [Co4] (29–60% de detecção) e [Co3].

5.3 Limitações centrais (leitura honesta)

1. **Poder estatístico restrito** ($n=12$; células esperadas <5): H_1 – H_3 são tendências, não conclusões.
2. **Pequena fração funcionalmente avaliável** (3/12 bootáveis): defeitos latentes em entregas não-bootáveis não puderam ser medidos.
3. **Condicionante de ambiente** (validação nativa, sem Docker): entregas configuradas apenas para contêiner foram tratadas como defeito de portabilidade; cuidamos de não confundir "defeito" com "suposição de ambiente".
4. **Classificação manual:** κ categoria moderado (0,54); resoluções registradas, mas a taxonomia sobre defeitos de integração permanece ambígua.
5. **Modelos "free" e momento da coleta:** os resultados descrevem esses modelos específicos na data da coleta; mudanças de rota/versão alteram o quadro.
6. **Amostra de defeitos pequena e viés Blocker** (por não-bootabilidade).
7. **Não-generalização:** um único sistema-alvo (tarefas) e stack (Python/FastAPI/PostgreSQL).

5.4 Síntese

No cenário controlado, agentes de modelos "free" em modo agente produziram sistemas completos com baixa taxa de execução (3/12), defeitos concentrados em poucos padrões reincidentes e nenhuma conformidade plena com o oráculo. A evidência mais sustentável é o **perfil de geração incompleta do ultra**; a mais geral é o **vale comum de qualidade** entre modelos gratuitos do mesmo pipeline. O laboratório exploratório, não evidenciário, reforça a separação entre *detectar* e *testar*. Os resultados **não** sustentam nem "IA não funciona" nem "IA substitui QA": sustentam que, **sob requisitos controlados e com oráculo independente, a qualidade entregue por esses agentes no momento da coleta ficou abaixo do mínimo funcional em 9 de 12 execuções**, e que a aprovação pelos próprios agentes não é substituta da validação independente.

6. Conclusão e Trabalhos Futuros

Conclusão. Este estudo contribui com um experimento controlado, prospectivo e reproduzível que combina, pela primeira vez — até onde a revisão alcança —, geração de sistemas completos por diferentes **agentes gratuitos de IA para código** a partir do mesmo requisito, múltiplas execuções, oráculo independente e avaliação simultânea de funcionalidade, defeitos, segurança, manutenibilidade, reprodutibilidade e testabilidade. Entre os resultados, destacamos: baixa executabilidade (3/12); perfil de defeitos concentrado em padrões reincidentes transferíveis entre modelos free ("vale comum"); e a **não-detecção dos defeitos pelos testes do próprio agente** (0/12) versus a detecção pelo oráculo (4/4 alcançáveis), reforçando que aprovação própria $\neq$ validação independente. Com o devido rigor de neutralidade, os dados — não uma premissa — sustentam essas leituras, dentro das limitações de poder estatístico e de generalização declaradas em §5.3.

Trabalhos futuros. (i) **Ampliação da amostra** (mais execuções e/ou execução em Docker na máquina de produção) para elevar células esperadas ≥ 5 e conferir poder ao χ^2 ; (ii) **execução dos contêineres** dos deliverable na máquina de produção (validação da camada Docker obrigatória na entrega); (iii) **extensão a mais modelos/famílias e a agentes completos distintos** (comparando também a ferramenta de agente, não só o modelo); (iv) formalizar **pré-registro** completo (já iniciado em repositório privado) para o desenho ampliado; (v) **estudo longitudinal** verificando se o "vale comum" persiste com atualizações dos modelos; (vi) **avaliação de autoria/qualidade da intervenção humana** mínima, contrastando com geração autônoma; (vii) **laboratórios pré-registrados** (EXP-003 e EXP-bridge) para testar a hipótese de faixa de validade de tipo/domínio e fronteiras, sem contaminação com o experimento formal; e (viii) uma **SLR formal** completa (a atual é intencionalmente enxuta e documentada como artefato de pesquisa).

7. Referências

IDs seguem a codificação da matriz de artigos da pesquisa. DOI verificados via Crossref/DataCite (16 periódico/conferência; 8 *preprints*; 1 relatório institucional sem DOI).

- [Ao1] HOU, X. et al. *Large Language Models for Software Engineering: A Systematic Literature Review*. ACM ToSEM, v. 33, n. 8, art. 220, 2024. DOI: [10.1145/3695988](https://doi.org/10.1145/3695988).
- [Ao2] UMAMA et al. *LLM-Based Code Generation: A Systematic Literature Review With Technical and Demographic Insights*. IEEE Access, v. 13, pp. 194915–194939, 2025. DOI: [10.1109/ACCESS.2025.3631952](https://doi.org/10.1109/ACCESS.2025.3631952).
- [Bo1] TAMBON, F. et al. *Bugs in Large Language Models Generated Code: An Empirical Study*. Empirical Software Engineering, v. 30, art. 65, 2025. DOI: [10.1007/s10664-025-10614-4](https://doi.org/10.1007/s10664-025-10614-4).

- [Bo2] DOU, S. et al. *What's Wrong with Your Code Generated by Large Language Models? An Extensive Study*. arXiv:2407.06153, 2024.
- [Bo3] COTRONEO, D.; IMPROTA, C.; LIGUORI, P. *Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity*. IEEE ISSRE 2025, pp. 252–263. DOI: 10.1109/ISSRE66568.2025.00035.
- [Bo4] OUYANG, S.; ZHANG, J. M.; HARMAN, M.; WANG, M. *An Empirical Study of the Non-Determinism of ChatGPT in Code Generation*. ACM ToSEM, v. 34, n. 2, art. 42, 2025. DOI: 10.1145/3697010.
- [Co1] WANG, J. et al. *Software Testing With Large Language Models: Survey, Landscape, and Vision*. IEEE TSE, v. 50, n. 4, pp. 911–936, 2024. DOI: 10.1109/TSE.2024.3368208.
- [Co2] BEER, A. et al. *Examination of Code generated by Large Language Models*. arXiv:2408.16601, 2024.
- [Co3] ALVES, V. A.; SANTOS, C.; BEZERRA, C. I. M.; MACHADO, I. *Detecting Test Smells in Python Test Code Generated by LLM: An Empirical Study with GitHub Copilot*. SBES 2024, CBSOFT. DOI: 10.5753/sbes.2024.3561.
- [Co4] YANG, L. et al. *On the Evaluation of Large Language Models in Unit Test Generation*. ASE 2024. DOI: 10.1145/3691620.3695529.
- [Co5] ZHAO, J.; ZHOU, S.; COHEN, E. *Do Coverage and Mutation Scores of LLM-Generated Test Suites Correlate with Their Effectiveness? (Replicability Study)*. ISSTA 2026. DOI: 10.1145/3832093.
- [Co6] WALCZAK, J.; TOMALAK, P.; LASKOWSKI, A. *Impact of code context and prompting strategies on automated unit test generation with modern general-purpose large language models*. Journal of Systems and Software, v. 237, art. 112834, 2026. DOI: 10.1016/j.jss.2026.112834.
- [Do1] TOSI, D. *Studying the Quality of Source Code Generated by Different AI Generative Engines: An Empirical Evaluation*. Future Internet, v. 16, n. 6, art. 188, 2024. DOI: 10.3390/fi16060188.
- [Do2] MOLISON, A. S. et al. *Is LLM-Generated Code More Maintainable & Reliable than Human-Written Code?*. arXiv:2508.00700, 2025.
- [Do3] BAUER, J.; LINDEMAN, L.; REILLY, L.; RODRIGUEZ, M. *Does GitHub Copilot improve code quality? Here's what the data says*. GitHub Customer Research, 2024. Disponível em: github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/.
- [Do4] SUN, X.; STÅHL, D.; SANDAHL, K.; KESSLER, C. *Quality assurance of LLM-generated code: Addressing non-functional quality characteristics*. Journal of Systems and Software, v. 238, art. 112885, 2026. DOI: 10.1016/j.jss.2026.112885.
- [Do5] LIU, Y. et al. *Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild*. arXiv:2603.28592, 2026.
- [Eo1] FU, Y. et al. *Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study*. ACM ToSEM, v. 34, n. 8, 2025. DOI: 10.1145/3716848.
- [Eo2] MORKONDA, S. G.; SELIM, M.; ASSAL, H. *Security of LLM-generated Code: A Comparative Analysis*. arXiv:2605.23091, 2026.
- [Eo3] KHARMA, M. et al. *An Empirical Evaluation of LLM-Generated Code Security Across Prompting Methods*. arXiv:2605.24298, 2026.
- [Fo1] HORIKAWA, K. et al. *Agentic Refactoring: An Empirical Study of AI Coding Agents*. arXiv:2511.04824, 2025.
- [Fo2] VANGALA, B. P.; ADIBIFAR, A.; GEHANI, A.; MALIK, T. *AI-Generated Code Is Not Reproducible (Yet): An Empirical Study of Dependency Gaps in LLM-Based Coding Agents*. arXiv:2512.22387, 2025/2026. DOI: 10.48550/arXiv.2512.22387.

- [Fo3] BELOZEROV, V.; BARCLAY, P. J.; SAMI, A. *Secure coding with AI – from detection to repair*. Empirical Software Engineering, v. 31, art. 93, 2026. DOI: 10.1007/s10664-026-10812-8.
- [Fo4] EHSANI, R. et al. *Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub*. MSR 2026, pp. 807–811. DOI: 10.1145/3793302.3793579.
- [Fo5] OUKHAY, I.; BEGOUG, M.; CHOUCHE, M.; OUNI, A. *When AI Code Doesn't Stick: An Empirical Study on Reverted Changes Introduced by AI Coding Agents*. MSR 2026, pp. 847–851. DOI: 10.1145/3793302.3793587.

Pré-print vo.1 – 2026-09-06.